

promptconf — Theory

promptconf v0.3.0

Abstract

This document formalizes `promptconf` v0.3.0: filesystem-native prompt versioning, pluggable backends, the version-resolution algebra (`latest`, tags, lock pins), dual template engines, compile caching, embedding search, deterministic A/B assignment, the closed-loop performance ledger, and lint-as-static-analysis. All claims track the public API in `promptconf.loader`, `backends`, `cache`, `ledger`, `vcs`, `template`, `ab`, and `lint`.

Contents

1	Problem statement	3
2	Store geometry	3
3	Version resolution algebra	3
3.1	Explicit version	3
3.2	<code>version="latest"</code> (floating alias)	3
3.3	Tags (named pointers)	4
3.4	Lock file (<code>freeze</code>)	4
3.5	Listing order	4
4	Pluggable backends	4
5	Template engines	5
5.1	<code>engine="format"</code> (default)	5
5.2	<code>engine="jinja"</code> (optional <code>[jinja]</code> extra)	5
5.3	Compile cache	5
5.4	Usage logging	6
6	A/B hashing	6
6.1	Deterministic assignment	6
6.2	Random assignment	6
7	Closed-loop performance ledger	6
7.1	Aggregation	6
7.2	Recommend	6
8	Search: keyword vs semantic	7
9	Lint as static analysis	7
10	Diff, CLI, and companions	7
11	Design invariants	8
12	Mapping to source	8

13 References**8**

1 Problem statement

Prompts are software artifacts: edited often, A/B tested, and easy to lose track of when inlined as string literals. `promptconf` provides an offline, git-friendly store:

$$\text{load}(n, v, x; R, e, B) \longrightarrow \text{string} \quad (1)$$

where n is the prompt name, v a version selector, x template variables, R the root, $e \in \{\text{format}, \text{jinja}\}$ the engine, and B an optional `PromptBackend`.

Root resolution precedence: explicit root $\rightarrow$ env `PROMPTCONF_ROOT` $\rightarrow$ `./prompts`. Bound API: `PromptStore(root=..., backend=...)`.

2 Store geometry

$R / n / \{v\}.\{\text{txt}, \text{md}, \text{prompt}\}$

- Prompt **names** = immediate subdirectories of R (skip `.-`-prefixed dirs).
- Version **stems** are file stems under $R/n/$.
- Extension priority when stems collide: `.txt` $\succ$ `.md` $\succ$ `.prompt`.
- Optional YAML frontmatter (`--- ... ---`) is stripped before render; `compile_prompt` exposes metadata + body.
- Empty / path-like names and empty versions $\rightarrow$ `ValidationError`.
- Files larger than `MAX_PROMPT_BYTES` (1 MiB, overridable via `max_bytes=`) $\rightarrow$ `PromptSizeError`.

Version stem classes:

1. Numeric releases matching `^v(\d+)$` (case-insensitive): `v1`, `v2`, `v10`.
2. Alias file stem `latest` (optional).
3. Arbitrary other stems (listed alphabetically between numerics and `latest`).

3 Version resolution algebra

Define $V(n)$ as the set of available stems for prompt n .

3.1 Explicit version

$$\text{resolve}(n, v) = \begin{cases} \text{path}(v) & \text{if } v \in V(n) \\ \perp & \text{otherwise (raises PromptNotFoundError)} \end{cases} \quad (2)$$

3.2 version="latest" (floating alias)

$$\text{resolve}(n, \text{latest}) = \begin{cases} \text{path}(\text{latest}) & \text{if } \text{latest} \in V(n) \\ \text{path}(v_{k^*}) & \text{if } k^* = \max\{k : v_k \in V(n)\} \\ \perp & \text{otherwise} \end{cases} \quad (3)$$

An explicit `latest.*` file **wins** over the highest numeric `vN`. This is intentional: `latest` is an alias, not “max semver.”

3.3 Tags (named pointers)

`tag(name, version, tag_name)` writes `{root}/.promptconf/tags.json`:

$$\text{tags}[\text{tag_name}] = \{\text{name} : n, \text{version} : v\} \quad (4)$$

`resolve_tag(tag_name)` returns that record. Tags are **not** auto-applied inside `load()`; callers resolve then `load(name, version=...)`. Validation ensures the target version exists before writing.

3.4 Lock file (freeze)

`freeze(root=...)` writes `{root}/prompt.lock.json` mapping each prompt name $\rightarrow$ a pin:

1. Prefer stem "latest" if present in $V(n)$,
2. Else highest numeric vN ,
3. Else last listed version.

Each pin is touch-resolved via `load(..., raw=True)` so broken pins fail at freeze time. The lock is a **reproducibility artifact** for CI and deploys; `load()` itself still takes an explicit `version=` (or "latest") — use lock pins as the version argument at call sites or in wrappers.

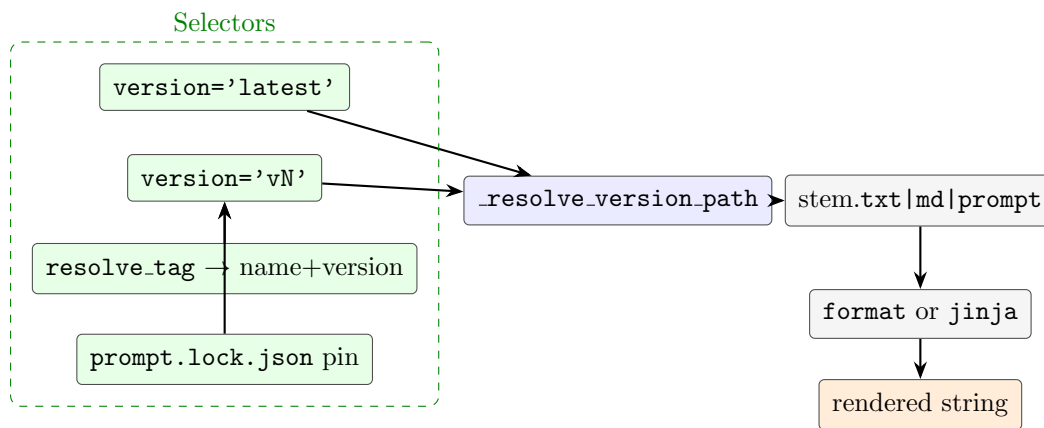


Figure 1: Version selectors funnel into path resolution, then template rendering. Tags and lock pins resolve externally to a concrete `version=` before `load`.

3.5 Listing order

Numeric v_k ascending $\rightarrow$ other non-`latest` stems alpha $\rightarrow$ `latest` last. Used by `list_versions` and error messages.

4 Pluggable backends

PromptBackend protocol:

FilesystemBackend. Default; wraps the geometry in Section 2.

GitStoreBackend. Treat a local git path (or clone URL + `cache_dir`) as prompt root; read from working tree; optional `commit` / `auto_commit`. Missing git CLI $\rightarrow$ `BackendUnavailableError` (stdlib `subprocess`; [git] extra documents the dependency).

`load(..., backend=B)` and `PromptStore(..., backend=B)` are additive. When `backend` is omitted, `load` uses the filesystem path directly; `PromptStore` defaults to `FilesystemBackend(root)`.

Method	Role
<code>list_prompts()</code>	Sorted names
<code>list_versions(name)</code>	Sorted stems
<code>read(name, version)</code>	Raw file text (incl. frontmatter)
<code>write(name, version, content)</code>	Optional; read-only backends may omit / error

Table 1: PromptBackend protocol methods.

5 Template engines

5.1 engine="format" (default)

Substitution via `str.format_map` / `string.Formatter`:

$$T(x) = \text{format}(T_0, x) \quad (5)$$

Mode	Behavior
<code>strict=True</code>	Missing root keys $\rightarrow$ <code>PromptFormatError</code>
<code>strict=False</code>	Missing keys left as literal <code>{key}</code>
<code>raw=True</code>	Skip formatting; return body after frontmatter strip

Table 2: Format-engine modes.

Root keys are the field names before `.` / `[` indexing. Fail-closed strict mode prevents silent quality regressions from incomplete instantiation.

5.2 engine="jinja" (optional [jinja] extra)

Rendering uses `jinja2.sandbox.SandboxedEnvironment` with `PromptIncludeLoader` rooted at `R`:

- Context variables: `{{ var }}`, control: `{% if %}`, macros, etc.
- Includes / extends resolve under the prompt root.
- `StrictUndefined` when `strict=True`; soft undefined preserves `{{ name }}` when not strict.
- Unsafe attributes (`__*__`, `mro`, frame/code attrs, etc.) blocked via custom `is_safe_attribute`.
- `SecurityError` / syntax / missing includes surface as `PromptFormatError` or `PromptNotFoundError`.

`compile_prompt` dry-runs parse validation without rendering (Jinja parse or `Formatter().parse` for format engine).

5.3 Compile cache

`CompileCache(dir)` stores rendered strings under:

$$k = \text{SHA256}(\text{canonical}(\text{content}, e, x_{\text{sorted}}, n, v)) \quad (6)$$

`load(..., use_cache=True)` / `load_cached(...)` hit the cache when `k` matches. Content edits change the hash $\rightarrow$ automatic invalidation (mtime alone is insufficient; bytes are authoritative). Default cache dir: `{root}/.promptconf/cache`.

5.4 Usage logging

When `log=True`, appends JSONL under `{root}/.promptconf/usage.jsonl` with **keys only** (never values) — reduces PII/secret leakage into the audit trail. `log(name)` (VCS helper) filters that file by prompt name.

6 A/B hashing

`ABRouter(experiments, root=..., seed=...)` holds per-prompt weight maps $\{v_i \mapsto w_i\}$ normalized to $\sum_i w_i = 1$.

6.1 Deterministic assignment

When `user_id` is provided:

$$b = \frac{\text{int}_{16}(\text{SHA256}(n:u)[:8])}{2^{32} - 1} \in [0, 1) \quad (7)$$

Then pick the first variant whose cumulative weight exceeds b (last variant is the residual). Sticky assignment: same (n, u) always maps to the same variant.

6.2 Random assignment

Without `user_id`, draw $b \sim U[0, 1)$ from an optional seeded `random.Random`.

`choose` may log an `ab.assignment` usage event; `load` chooses then calls `promptconf.loader.load` for that version.

7 Closed-loop performance ledger

Differentiator. Pins, tags, and A/B logs describe *what ran*. The ledger answers *which prompt wins in production*.

Outcomes append to `{root}/.promptconf/outcomes.jsonl`:

$$o_t = (n, v, m_k, m, u, \dots) \quad (8)$$

where m_k defaults to "reward" and $m \in \mathbb{R}$ is an external metric (task success, thumbs-up, latency inverse, ...).

7.1 Aggregation

$$\text{best_version}(n; m_k, s_{\min}) = \underset{v: |O_{n,v,m_k}| \geq s_{\min}}{\text{argmax}} \quad \text{mean}(O_{n,v,m_k}) \quad (9)$$

Ties break by larger sample count, then version label. Returns $\emptyset$ when no version meets $s_{\min}$.

7.2 Recommend

$$\text{recommend}(n, u) = \begin{cases} \text{best_version}(n) & \text{if defined} \\ \text{ABRouter.choose}(n, u) & \text{otherwise} \end{cases} \quad (10)$$

This closes the loop: ship weighted variants $\rightarrow$ ingest outcomes $\rightarrow$ promote winners without a cloud prompt hub.

Public surface: `record_outcome`, `best_version`, `recommend`, and bound helper `PerformanceLedger(root=, router=...)`. Outcomes live at `{root}/.promptconf/outcomes.jsonl` (separate from `usage.jsonl`).

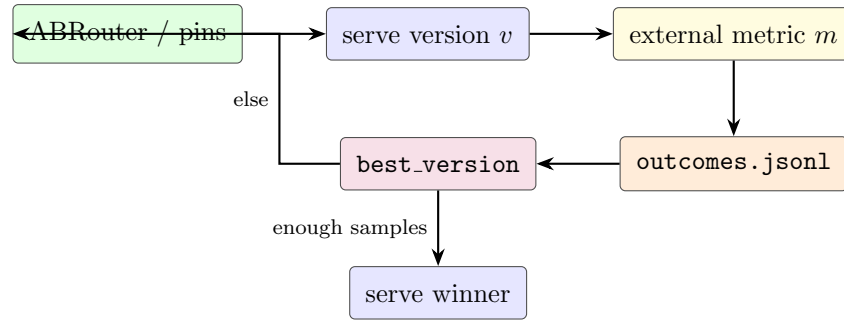


Figure 2: Closed-loop ledger: A/B cold-start, metric ingest, winner promotion.

8 Search: keyword vs semantic

- `search(query)` — substring / regex over bodies; `SearchHit.score` denser/earlier matches.
- `semantic_search(query, top_k=...)` — embed query + bodies; rank by cosine similarity; same `SearchHit` shape with cosine `score`.
 - Default embedder: `HashEmbedder` (deterministic bag-of-words hashing; offline/tests).
 - Production: `OpenAIEmbedder` behind `[embeddings]`.

Keyword `search()` is unchanged.

9 Lint as static analysis

Prompt lint is **static**: it inspects text without calling an LLM. `LintRules` / `lint_prompt` / `lint_all` emit `LintIssue` records (`rule`, `message`, `severity`, `location`).

Rule	Idea
<code>max_chars</code>	Length budget (error)
<code>banned_phrases</code>	Case-insensitive substring denylist (error)
<code>require_placeholders</code>	Must contain <code>{name}</code> format fields (error)
<code>no_trailing_whitespace</code>	Line hygiene (warning)
<code>json_mode_hint</code>	Regex detect JSON-output language: warn / require / forbid

Table 3: Static lint rules.

This is analogous to a linter AST pass: cheap, deterministic, CI-friendly. Schema validation (`validate_vars` / `load_validated`) is a related static check over declared variable contracts, separate from prose lint rules.

10 Diff, CLI, and companions

Conceptual companions (same package):

- `diff_versions` — textual diff across versions for review.
- CLI: `diff` / `tag` / `freeze` / `log` / lint- oriented workflows.

These do not change resolution algebra; they operationalize it for humans and CI.

11 Design invariants

1. **Filesystem (or backend) is the registry** — Git distributes; no cloud hub required.
2. **latest is an alias, not $\max(vN)$ when a latest file exists.**
3. **Tags and locks are pointers** — resolve externally, then load by concrete version.
4. **Strict formatting fails closed** by default.
5. **Jinja is sandboxed** — treat templates as untrusted when variables or includes are attacker-influenced.
6. **Usage logs never store variable values.**
7. **Ledger promotes winners from production metrics** — A/B is the cold-start fallback.
8. **Backends fail loud** when unavailable (`BackendUnavailableError`).
9. **Oversized prompts fail loud** (`PromptSizeError` / `MAX_PROMPT_BYTES`).
10. **Invalid names/versions fail loud** (`ValidationError`, also a `ValueError`).

12 Mapping to source

Concept	Module
Root, load, resolve, compile, <code>MAX_PROMPT_BYTES</code>	<code>promptconf.loader</code>
Bound store	<code>promptconf.store.PromptStore</code>
Backends	<code>promptconf.backends</code> (<code>FilesystemBackend</code> , <code>GitStoreBackend</code>)
Compile cache	<code>promptconf.cache</code> (<code>CompileCache</code> , <code>load_cached</code>)
Performance ledger	<code>promptconf.ledger</code> (<code>record_outcome</code> , <code>best_version</code> , <code>recommend</code>)
Semantic search	<code>promptconf.semantic_search</code>
Exceptions	<code>promptconf.exceptions</code> (<code>ValidationError</code> , <code>BackendUnavailableError</code> , <code>PromptSizeError</code> , ...)
Tags, freeze, usage log read	<code>promptconf.vcs</code>
Jinja sandbox	<code>promptconf.template</code>
Includes loader	<code>promptconf.includes</code>
A/B router	<code>promptconf.ab</code>
Lint	<code>promptconf.lint</code>
Frontmatter	<code>promptconf.meta</code>
Schema validation	<code>promptconf.schema</code>

Table 4: Concept-to-module mapping.

13 References

- [1] Liu, P., et al. (2021). *Pre-train, Prompt, and Predict*. <https://arxiv.org/abs/2107.13586>
- [2] Khattab, O., et al. (2023). *DSPy*. <https://arxiv.org/abs/2310.03714>

- [3] Schulhoff, S., et al. (2024). *The Prompt Report*. <https://arxiv.org/abs/2406.06608>
- [4] Jinja2 SandboxedEnvironment documentation (template isolation).
- [5] Python `string.Formatter` (format-engine field parsing).
- [6] OpenAI Embeddings API (optional semantic search).